BÖLÜM 1 / MODULE 1

Thread'ler, İşletim Sistemi Çekirdeği & JMM Temelleri

Threads, OS Kernels & Java Memory Model (JMM) Fundamentals

Process vs Thread farkı, JVM thread durumları, donanım önbellekleri (L1/L2/L3), False Sharing, JMM Happens-Before garantisi ve modern Java 25 thread oluşturma mekanizmaları.

Process vs Thread dichotomy, JVM thread states, CPU cache hierarchies (L1/L2/L3), False Sharing, the JMM Happens-Before relationship, and modern Java 25 thread creation.

1

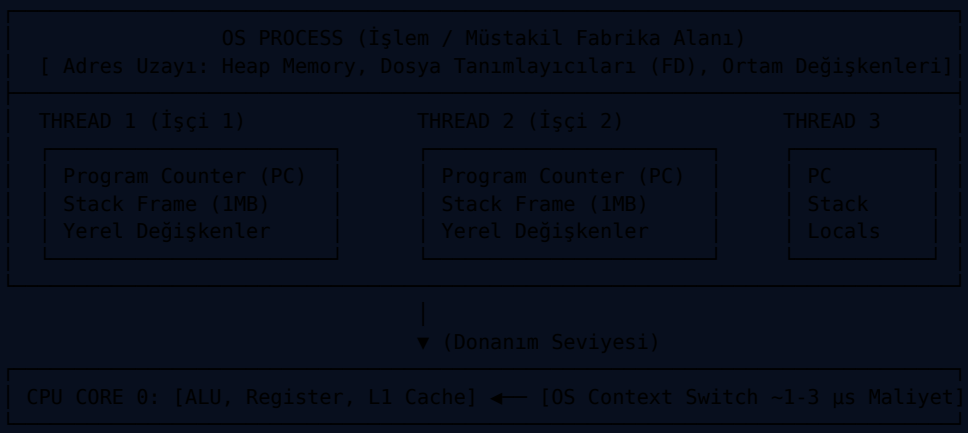
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Nedir? Process vs Thread, OS Context Switching & CPU Çekirdekleri

What is a Thread? Process vs Thread, OS Context Switching & CPU Cores

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir Process (İşlem), etrafı yüksek duvarlarla çevrili koca bir fabrikadır. Kendi su tesisatı, elektrik sayacı ve tapusu (ayrı RAM adres alanı) vardır. Bir fabrikanın diğer fabrikanın odalarına izinsiz girmesi yasaktır! Thread (İş parçacığı) ise o fabrikanın içinde koşturan işçilerdir. Bütün işçiler aynı fabrikanın ortak deposunu (Heap Memory) paylaşırlar; ama her işçinin cebinde kendi özel not defteri (Thread Stack) vardır. İki işçi aynı depodaki aynı kutuya (Paylaşılan Değişken) haberleşmeden aynı anda dalarsa ortalık karışır (Race Condition)!

🇬🇧 ENGLISH (ELI5)

A Process is an entire secure factory compound with its own private land, utilities, and security gate (isolated virtual address space). Process A cannot inspect Process B's floor plan. A Thread is a worker inside that factory. All workers walk the same shared factory warehouse floor (Heap Memory), but each worker carries a personal pocket notepad (Thread Stack). If two workers update the same inventory box simultaneously without coordination, inventory records corrupt instantly (Race Condition)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG seviyesinde Process vs Thread anatomisi: 1) Bellek İzolasyonu: Process'ler MMU (Memory Management Unit) ve Page Table ile sanal adres alanlarında tamamen izole edilir. İletişim IPC (Inter-Process Communication: Sockets, Shared Memory, Pipes) gerektirir ve pahalıdır. 2) Thread Paylaşımı: Bir JVM process'indeki tüm platform thread'leri Heap, Metaspace ve dosya tanıtıcılarını (FD) paylaşır; ancak her thread kendi Stack alanına (1MB varsayılan), Program Counter (PC) register'ına ve Native Method Stack'ine sahiptir. 3) Context Switch Maliyeti: İşletim sistemi çekirdeği (Kernel) iki platform thread'i arasında geçiş yaparken CPU register'larını kaydeder, TLB (Translation Lookaside Buffer) önbelleğini temizler ve L1/L2 CPU cache'lerini bozar (Cache Pollution). Bu maliyet 1-3 mikrosaniyedir ve saniyede milyonlarca istek alan MAANG sistemlerinde verim kaybına yol açar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG-level architectural mechanics: 1) Address Space Isolation: Processes operate in distinct virtual memory boundaries governed by the hardware MMU and OS page tables. Cross-process IPC requires context-switching into ring-0 kernel space. 2) Intra-Process Thread Concurrency: Java platform threads map 1:1 to OS native threads (POSIX pthreads / Windows kernel threads). They share the JVM Heap and Metaspace, but retain private Program Counters and runtime call stacks (1MB default). 3) Context Switching Overhead: When the OS scheduler preempts Thread A for Thread B, it saves instruction pointers and registers to kernel memory, often triggering TLB invalidation and L1/L2 cache misses (Cache Pollution), incurring 1-3 microseconds of pure CPU latency per switch.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Platform Thread vs Donanım Çekirdekleri Bilgisi
void main() {
    int availableCores = Runtime.getRuntime().availableProcessors();
    long totalMemoryMb = Runtime.getRuntime().totalMemory() / (1024 * 1024);

    IO.println("CPU Donanım Çekirdek Sayısı: " + availableCores);
    IO.println("JVM Tahsis Edilen Heap Bellek: " + totalMemoryMb + " MB");

    // Geleneksel Platform Thread başlatma
    Thread osThread = new Thread(() -> {
        var current = Thread.currentThread();
        IO.println("Çalışan İşçi: " + current.getName() + " | Platform Thread mi: " + !
current.isVirtual());
    }, "MAANG-Worker-01");

    osThread.start();
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Daha fazla thread açarsak sistemimiz her zaman daha mı hızlanır?' diye sorar. Yanıt: Hayır! CPU çekirdek sayısından çok fazla platform thread açılırsa, CPU zamanını iş yapmaya değil işçiler arasında geçiş yapmaya (Context Switching thrashing) harcar ve throughput çöker!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Interview Trap: The interviewer asks: 'Can we endlessly boost throughput simply by adding more threads?' Senior Answer: 'No! If runnable platform thread count drastically exceeds physical CPU cores, the kernel spends more clock cycles on thread context switching and cache invalidation than actual task computation, precipitating throughput collapse.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Process'ler izole adres alanlarına sahiptir; Thread'ler ise aynı Heap alanını paylaşır.
- Platform thread'leri 1:1 işletim sistemi kernel thread'lerine eşlenir (~1MB stack).
- Gereğinden fazla thread context-switching thrashing'e yol açarak sistemi kilitler.

🎯 Key Takeaways for the Interview

- Processes own isolated virtual address spaces; threads share the JVM heap.
- Platform threads map 1:1 to kernel threads, incurring ~1MB memory footprint.
- Oversubscribing threads degrades throughput via context-switch thrashing.

2

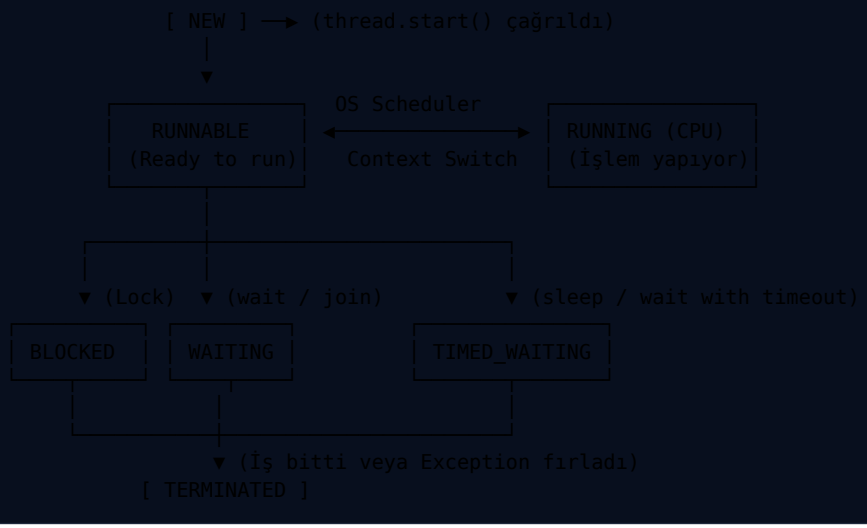
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Java Thread Yaşam Döngüsü: 6 Kritik Durum (State Transitions)

Java Thread Lifecycle: The 6 Core JVM States & State Transitions

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir işçinin gün içindeki 6 hali vardır: 1) NEW: İşe yeni alındı, masasında oturuyor ama henüz 'Başla' denmedi. 2) RUNNABLE: Masasında hazır bekliyor veya harıl harıl çalışıyor. 3) BLOCKED: Tuvalet kapısının önünde bekliyor çünkü içeride biri var (Kilit kavgası). 4) WAITING: Patron 'Ben sana haber verene kadar kenarda bekle' dedi (Süresiz bekleme). 5) TIMED_WAITING: '20 dakika öğle uykusuna yat' dendi (Süreli bekleme). 6) TERMINATED: Mesaisi bitti, evine gitti!

🇬🇧 ENGLISH (ELI5)

An employee's lifecycle stages: 1) NEW: Hired and given a badge, but hasn't clocked in yet. 2) RUNNABLE: Either actively typing at the desk or standing ready for an assignment. 3) BLOCKED: Waiting outside the restroom because a colleague locked the door from inside (Lock Contention). 4) WAITING: Sitting in the lounge waiting indefinitely for a manager's signal. 5) TIMED_WAITING: Taking an exact 15-minute coffee break. 6) TERMINATED: Shift finished, clocked out forever!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Thread.State enum değerleri ve mülakat derinliği: 1) NEW: `new Thread()` yapılmış ancak `start()` çağrılmamıştır. `start()` yerine doğrudan `run()` çağrılırsa yeni thread açılmaz, çağıran thread içinde senkron çalışır! 2) RUNNABLE: JVM seviyesinde thread koşmaya hazırdır veya CPU'da çalışıyordur. JVM, işletim sisteminin 'Ready' ve 'Running' durumlarını tek bir RUNNABLE altında toplar. 3) BLOCKED: Thread bir `synchronized` bloğuna girmek için Monitor Lock (Intrinsic Lock) beklemektedir. 4) WAITING: `Object.wait()`, `Thread.join()` veya `LockSupport.park()` ile süresiz askıya alınmıştır; başka bir thread `notify()` veya `unpark()` çağırana kadar CPU tüketmez. 5) TIMED_WAITING: `Thread.sleep(ms)`, `wait(ms)` veya `parkNanos()` ile zaman sınırıyla bekler. 6) TERMINATED: `run()` metodu doğal yolla bitmiş veya yakalanmamış bir exception ile sonlanmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Thread State nuances: 1) NEW to RUNNABLE: Calling `start()` triggers OS `pthread_create()`. Invoking `run()` directly bypasses thread creation, executing synchronously on the caller's stack. 2) RUNNABLE Collapsing: The JVM intentionally does not distinguish between OS 'Ready' (in kernel runqueue) and OS 'Executing' (on CPU core); both report as RUNNABLE. 3) BLOCKED vs WAITING: A critical distinction in thread dumps! BLOCKED strictly means waiting for a synchronized monitor lock. WAITING means parked via `LockSupport.park()` or `Object.wait()`, awaiting a signal. 4) TERMINATED: Dead threads cannot be restarted. Calling `start()` on a TERMINATED thread throws `IllegalThreadStateException`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Thread Durum Geçişlerini İnceleme
void main() throws InterruptedException {
    final Object lock = new Object();

    Thread worker = new Thread(() -> {
        try {
            Thread.sleep(100); // TIMED_WAITING
            synchronized (lock) {
                lock.wait();   // WAITING
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });

    IO.println("1. Durum: " + worker.getState()); // NEW
    worker.start();
    IO.println("2. Durum: " + worker.getState()); // RUNNABLE

    Thread.sleep(50);
    IO.println("3. Durum (Sleep anı): " + worker.getState()); // TIMED_WAITING

    Thread.sleep(100);
    synchronized (lock) {
        IO.println("4. Durum (Wait anı): " + worker.getState()); // WAITING
        lock.notify();
    }

    worker.join();
    IO.println("5. Durum (Bitti): " + worker.getState()); // TERMINATED
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'ReentrantLock bekleyen bir thread hangi duruma geçer?' Yanıt: BLOCKED DEĞİL, WAITING (veya TIMED_WAITING) durumuna geçer! Çünkü ReentrantLock `LockSupport.park()` kullanır; BLOCKED durumu SADECE `synchronized` monitor lock için geçerlidir!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Interview Trap: 'What state is a thread in when blocked on a ReentrantLock?' Answer: 'WAITING (or TIMED_WAITING), NOT BLOCKED! ReentrantLock is built on AbstractQueuedSynchronizer (AQS) using LockSupport.park(). BLOCKED is strictly reserved for intrinsic synchronized monitors.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- JVM'de 6 thread durumu vardır: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED sadece synchronized içindir; ReentrantLock bekleyen thread WAITING olur.
- Ölmüş (TERMINATED) bir thread yeniden `start()` edilemez.

🎯 Key Takeaways for the Interview

- JVM threads traverse 6 states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- BLOCKED is exclusive to synchronized; AQS locks park threads in WAITING state.
- A TERMINATED thread cannot be restarted; doing so throws IllegalThreadStateException.

3

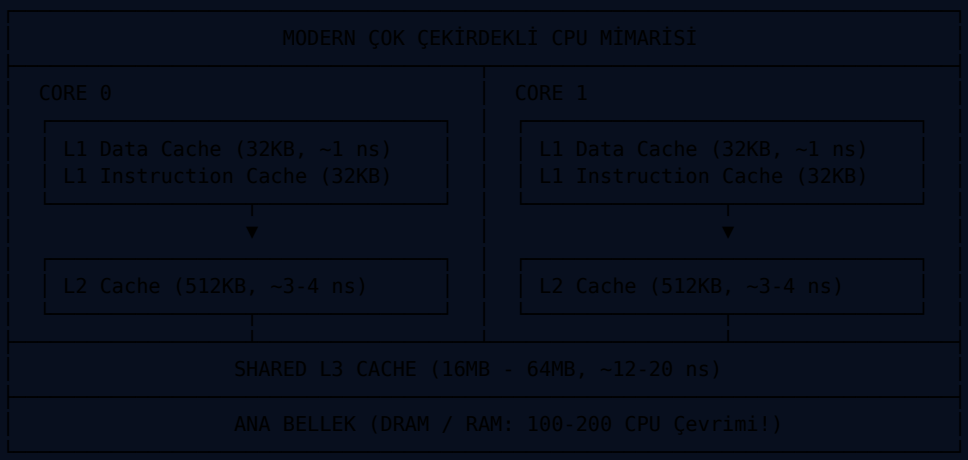
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Donanım Mimarisi: CPU Önbellekleri (L1/L2/L3), Store Buffer & False Sharing

Hardware Reality: CPU Caches (L1/L2/L3), Store Buffers & False Sharing

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

CPU o kadar hızlı koşar ki, ana belleğe (RAM) gitmek onun için 3 ay süren bir gemi yolculuğu gibidir! Bu yüzden masasının hemen üstüne minicik çekmeceler koymuştur: L1 (kol mesafesinde), L2 (odanın köşesinde), L3 (koridorda). CPU veriyi belleğe giderken tek bir bayt olarak değil, her defasında 64 baytlık paket (Cache Line) halinde getirir. Eğer iki farklı işçi, aynı 64 baytlık paketin içindeki farklı iki kutuyu güncellemeye çalışırsa, CPU'lar sürekli birbirinin çekmecesini geçersiz kılar (False Sharing). İşçiler birbirine dokunmasa bile sistem yerlerde sürünür!

🇬🇧 ENGLISH (ELI5)

CPUs execute so fast that fetching from RAM feels like waiting 3 months for cargo delivery! To survive, the core keeps micro-drawers on its desk: L1 (arm's length, 1ns), L2 (across the room, 4ns), L3 (down the hall, 15ns). Hardware never transfers a lone 4-byte integer from RAM; it fetches an entire 64-byte Cache Line. If Thread A on Core 0 modifies variable X, and Thread B on Core 1 modifies variable Y, but X and Y sit inside the same 64-byte line, cache coherency protocols (MESI) invalidate both cores' caches repeatedly (False Sharing)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MESI Protokolü ve False Sharing derinliği: 1) Cache Line: Modern x86 ve ARM64 mimarilerinde önbellek aktarım birimi 64 bayttır. 2) MESI Protokolü (Modified, Exclusive, Shared, Invalid): Bir çekirdek bir cache line'a yazdığında, donanım veri yoluna 'Invalidate' sinyali gönderir. Diğer tüm çekirdeklerin o satırdaki L1/L2 kopyaları anında çöpe gider ve RAM/L3'ten tekrar çekilmek zorunda kalır. 3) False Sharing Felaketi: Mantıksal olarak bağımsız iki `volatile` veya atomik değişken bellekte bitişik tanımlanmışsa (ör. `long a; long b;`), farklı thread'ler bunlara bağımsız yazsa bile aynı 64 baytlık satır çekirdekler arasında pinpon topu gibi gidip gelir (Bus saturation). 4) Çözüm: Java 8+ ile gelen `\@jdk.internal.vm.annotation.Contended` anotasyonu veya manuel alan doldurma (Padding: 128 bayt aralık bırakma).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Cache Coherency & False Sharing mechanics: 1) The 64-Byte Unit: Memory subsystems always populate L1/L2 caches in contiguous 64-byte aligned blocks. 2) MESI Protocol State Flips: When Core 0 writes to an address, it transitions the cache line from Shared to Modified, forcing Core 1 to mark its local copy as Invalid. 3) False Sharing Inefficiency: If Thread 1 frequently writes to `val1` and Thread 2 writes to `val2`, and both fields inhabit the same 64-byte cache line, the CPU interconnect is choked with cache-line invalidation traffic despite zero algorithmic overlap. 4) Remediation: JVM `\@Contended` annotation (requires `-XX:-RestrictContended`) pads instances with 128 dummy bytes, ensuring isolated cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: False Sharing Korumalı Cache-Line Padded Sayaç
public class CachePaddedCounter {
    // 64-baytlık satırı doldurmak için boşluk (Padding) bırakma tekniği
    public static class PaddedAtomicLong {
        // Nesne başlığı (Object Header) ~12-16 byte
        // Önceki 56 bayt doldurma
        public volatile long p1, p2, p3, p4, p5, p6, p7;

        // Asıl değerimiz (8 bayt)
        public volatile long value = 0L;

        // Sonraki 56 bayt doldurma
        public volatile long p8, p9, p10, p11, p12, p13, p14;
    }

    void main() {
        var counter = new PaddedAtomicLong();
        counter.value = 42L;
        IO.println("Cache-line ayrılmış güvenli sayaç değeri: " + counter.value);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'Dizideki farklı indekslere farklı thread'ler yazarsa race condition olur mu?' diye sorar. Yanıt: 'Mantıksal race condition olmaz ama dizinin bitişik elemanları aynı 64-baytlık cache line'a düşeceği için donanımsal False Sharing felaketi yaşanır ve performans 50 kat düşebilir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

High-Impact Interview Trap: 'If Thread A writes to array[0] and Thread B writes to array[1], is there any concurrency penalty?' Senior Answer: 'There is no semantic data race, but there is severe False Sharing. Because array[0] and array[1] share a single 64-byte cache line, CPU cores continually invalidate each other's L1 caches, slashing throughput by up to 50x!'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CPU veriyi RAM'den 64 baytlık Cache Line blokları halinde transfer eder.

• False Sharing, bağımsız değişkenlerin aynı cache satırını paylaşmasıyla oluşur.

• Yüksek performanslı kuyruklar (Disruptor) ve sayaçlar (LongAdder) cache padding kullanır.

🎯 Key Takeaways for the Interview

• CPUs fetch memory in atomic 64-byte cache lines, not single primitive bytes.

• False Sharing occurs when unrelated variables occupy the same 64-byte cache line.

• High-throughput engines (e.g. LMAX Disruptor) pad memory to isolate cache lines.

4

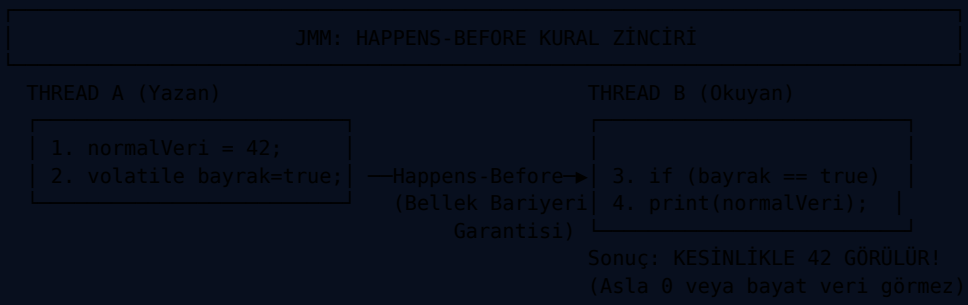
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Java Memory Model (JMM): Happens-Before Garantisi Nedir?

Java Memory Model (JMM): The Happens-Before Relationship Decoded

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



TEMEL HAPPENS-BEFORE KURALLARI:
1. Program Order Rule: Aynı thread içindeki eylemler yazılış sırasıyla gerçekleşir.
2. Monitor Lock Rule: Bir kilidin bırakılması (unlock), sonraki kilitlemeden (lock) önce gelir.
3. Volatile Variable Rule: Bir volatile yazma, sonraki volatile okumadan önce gelir.
4. Thread Start Rule: thread.start() çağrısı, thread içindeki ilk eylemden önce gelir.

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İki kişi farklı odalarda kağıda yazı yazıyor olsun. Biri yazdığı kağıdı çekmeceye koymadan (RAM'e basmadan) öbürü o kağıdı göremez. 'Happens-Before' kuralı, iki yabancı işçi arasındaki yasal noter sözleşmesidir: 'A işçisi bayrağı kaldırdığı anda, o bayrağı kaldırmadan önce masaya koyduğu BÜTÜN evraklar B işçisine görünür olmak zorundadır!' Eğer bu yasal sözleşme (Happens-Before) yoksa, B işçisi bayrağı kalkmış görse bile evrakları hala bomboş (0 veya null) görebilir!

🇬🇧 ENGLISH (ELI5)

Imagine two clerks working in separate soundproof rooms. Clerk A drafts a contract, then rings a service bell. The Happens-Before guarantee is the legal protocol ensuring that when Clerk B hears that bell, ALL documents Clerk A drafted prior to ringing MUST be visibly sitting in the shared transfer tray. Without a formal Happens-Before edge, Clerk B might hear the bell but open the tray to find blank paper because Clerk A's edits are still stuck in their private desk drawer (Store Buffer)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

JSR-133 ve JMM Spesifikasyonu: 1) Görünürlük Sorunu (Visibility): Modern CPU'lar yazma işlemlerini hemen ana belleğe yazmaz; Store Buffer'a atar. Happens-Before ilişkisi kurulmazsa, Thread B değişkenin güncel halini dakikalarca göremeyebilir. 2) 8 Temel Happens-Before Kuralı: a) Program Order: Tek thread içi ardışıklık. b) Monitor Lock: `unlock` -> sonraki `lock`. c) Volatile Variable: `volatile write` -> sonraki `volatile read`. d) Thread Start: `t.start()` -> `t` içindeki kodlar. e) Thread Join: t'nin bitişi -> `t.join()` dönüşü. f) Interruption: `interrupt()` -> kesintiyi tespit eden thread. g) Finalizer: Constructor bitişi -> `finalize()` başlangıcı. h) Transitivity (Geçişlilik): Eğer A -> B ve B -> C ise, A -> C'dir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

JSR-133 JMM Formalism: 1) The Memory Visibility Illusion: CPUs utilize out-of-order execution, store buffers, and invalidation queues. Memory operations do not occur globally in instant lockstep. 2) Formal Definition of Happens-Before (HB): An execution order defined over actions. If action X happens-before action Y, then the memory mutations performed by X are guaranteed visible to Y, and X is ordered before Y. 3) Transitivity Property: If action A happens-before B (via program order), and action B happens-before C (via volatile write-read or lock release-acquire), then A transitively happens-before C, safely publishing all non-volatile state preceding B.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Happens-Before Transitivite Güvencesi
public class HappensBeforeDemo {
    private int regularData = 0;
    private volatile boolean ready = false;

    public void writerThread() {
        regularData = 100; // 1. Normal yazma
        ready = true;      // 2. Volatile yazma (Happens-Before Bariyeri kurar!)
    }

    public void readerThread() {
        if (ready) { // 3. Volatile okuma
            // JMM Garantisi: ready=true ise regularData KESİNLİKLE 100'dür!
            IO.println("Veri güvenle okundu: " + regularData);
        }
    }

    void main() throws InterruptedException {
        var demo = new HappensBeforeDemo();
        Thread t1 = new Thread(demo::writerThread);
        Thread t2 = new Thread(demo::readerThread);

        t1.start();
        t1.join();
        t2.start();
        t2.join();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Ölümcül Hata: Happens-Before kurulmazsa, derleyici ve CPU kodun sırasını değiştirebilir! Yani `ready = true` işlemi `regularData = 100` işleminden ÖNCE hafızaya basılabilir! Okuyucu thread `ready == true` görür ama `regularData` hala `0` çıkar!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Fatal Interview Trap: Without an explicit Happens-Before edge, the JIT compiler and hardware can reorder instructions! The CPU might publish `ready = true` to shared memory BEFORE `regularData = 100` drains from the store buffer. The reader sees `ready == true` but reads `regularData == 0`!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Happens-Before, bir thread'in yazmasının diğerine ne zaman görüneceğini garanti eder.
- Volatile yazma, kendisinden önceki tüm normal yazmaları da görünür kılar (Transitivity).
- Happens-Before yoksa derleyici ve CPU talimatları rastgele yeniden sıralayabilir.

🎯 Key Takeaways for the Interview

- Happens-Before formally dictates when mutations by Thread A become visible to Thread B.
- A volatile write flushes all preceding non-volatile writes via transitive happens-before.
- Without happens-before, hardware and JIT aggressively reorder memory instructions.

5

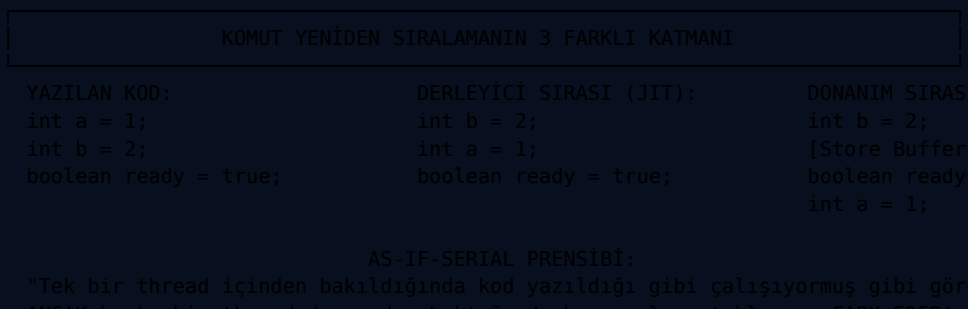
BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Komut Yeniden Sıralama (Instruction Reordering) & Derleyici Optimizasyonları

Instruction Reordering & Compiler Optimizations: Why Code Runs Out of Order

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir aşçı düşünün: Tarifte '1. Havuçları soy, 2. Eti marine et, 3. Tencereye yağ dök' yazıyor. Akıllı aşçı bakar ki yağın kızması 3 dakika sürüyor; önce yağı ocağa koyar, o ısınırken havuçları soyar! Tek başına mutfakta olduğu sürece yemek aynı lezzette çıkar (As-If-Serial). Ama mutfağa aniden ikinci bir çırak girip 'Yağ ocakta olduğuna göre havuçlar çoktan doğranmıştır' diye tencereye boş elini atarsa eli yanar! Çoklu thread dünyasında bu sıralama hileleri felakete yol açar!

🇬🇧 ENGLISH (ELI5)

Consider a master chef reading a recipe: '1. Peel carrots, 2. Season steak, 3. Heat oil in skillet'. Because heating oil takes 4 minutes, the chef turns on the stove burner first, then peels carrots while the pan heats up. To the chef working alone, the final meal tastes identical (As-If-Serial rule). But if an assistant rushes in and assumes 'The stove is hot, therefore chopped carrots must already be prepared!', disaster strikes. In multithreading, reordering tricks destroy uncoordinated observers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç Seviyeli Yeniden Sıralama: 1) Derleyici Yeniden Sıralaması (Compiler Reordering): javac ve HotSpot JIT (C1/C2), register tahsisini ve döngü verimliliğini optimize etmek için bağımsız talimatların yerini değiştirir. 2) İşlemci Yeniden Sıralaması (CPU Out-of-Order Execution): Modern süperskalar CPU'lar boru hattını (Pipeline) dolu tutmak için bağımsız yönergeleri eşzamanlı veya sıra dışı yürütür. 3) Bellek Sistemi Sıralaması (Memory Hierarchy Reordering): Store Buffer ve Invalidation Queue'lar yüzünden yazma işlemleri hemen RAM'e düşmez; okuma işlemleri bekleyen yazmaların önüne geçebilir (StoreLoad reordering). 4) As-If-Serial Semantiği: JVM ve donanım, tek thread içinde veri bağımlılığı (Data Dependency: Read-After-Write, Write-After-Read, Write-After-Write) olan komutların sırasını ASLA bozmaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Three Tiers of Reordering: 1) Compiler Reordering: HotSpot C2 JIT reorders instructions to maximize register utilization and eliminate redundant loads. 2) Processor Reordering: Superscalar architectures employ reservation stations and out-of-order execution pipelines to avoid stall cycles during cache misses. 3) Memory Subsystem Reordering: Store Buffers allow the CPU to continue processing while memory writes are lazily drained, allowing subsequent loads to bypass stale stores. 4) The As-If-Serial Invariant: Compilers and silicon pledge that single-threaded execution behavior is indistinguishable from strict serial execution. However, they make ZERO guarantees to concurrent observer threads without explicit memory fences.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Instruction Reordering Kanıtı (jcstress Mantığı)
public class ReorderingTest {
    private int x = 0, y = 0;
    private int r1 = 0, r2 = 0;

    // Thread 1 çalıştırır:
    public void actor1() {
        x = 1;   // Yazma 1
        r1 = y;  // Okuma 1
    }

    // Thread 2 çalıştırır:
    public void actor2() {
        y = 1;   // Yazma 2
        r2 = x;  // Okuma 2
    }

    // Olası Sonuçlar:
    // r1=0, r2=1 (Thread 1 önce bitti)
    // r1=1, r2=0 (Thread 2 önce bitti)
    // r1=1, r2=1 (İki yazma da önce bitti)
    // r1=0, r2=0 (İMKANSIZ GİBİ GÖRÜNÜR AMA REORDERING YÜZÜNDEN GERÇEKLEŞİR!)
    void main() {
        IO.println("Instruction Reordering: Donanım StoreLoad taklası r1=0, r2=0 sonucunu
doğurabilir!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Sorusu: 'Tek bir thread içinde `x = 1; y = 2;` yazarsak CPU hangisini önce çalıştırır?' Doğru Yanıt: 'Aralarında veri bağımlılığı olmadığı için CPU veya JIT dilediğini önce çalıştırabilir! Tek thread için sonuç fark etmez ama başka bir thread araya girerse patlar.'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Sharp Interview Question: 'In `x = 1; y = 2;`, does the JVM guarantee x is written before y?' Correct Answer: 'No! There is zero data dependency between x and y. The compiler or CPU can flip their execution order freely. To enforce ordering across threads, memory barriers (`volatile` or locks) are mandatory.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- As-if-serial kuralı sıralama garantisini yalnızca tek bir thread için verir.
- Modern CPU'lar bellek gecikmesini gizlemek için komutları sıra dışı (out-of-order) çalıştırır.
- Farklı thread'ler arasındaki sıralama yalnızca Memory Barrier (Fences) ile zorlanabilir.

🎯 Key Takeaways for the Interview

- The as-if-serial contract guarantees preservation of program semantics strictly for single threads.
- Hardware speculatively reorders instructions out-of-order to hide memory latency.
- Cross-thread instruction ordering requires explicit hardware memory barriers.

6

BÖLÜM: THREAD'LER, İŞLETIM SISTEMI ÇEKIRDEĞI & JMM TEMELLERI | MODULE: THREADS, OS KERNELS & JAVA MEMORY MODEL (JMM) FUNDAMENTALS

Thread Oluşturma Paradigmaları: Thread, Runnable, Callable & Java 25 Builder

Thread Creation Paradigms: Thread, Runnable, Callable & Modern Java 25 Builders

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
1. ESKİ USUL (Kalıtım / Kötü):
   class Worker extends Thread { public void run() { ... } }
   ❌ Java tekli kalıtımını harcar, Thread yaşam döngüsü ile iş mantığı birbirine karışır!

2. AYRIŞTIRILMIŞ İŞ PARÇACIĞI (Composition / Standart):
   Runnable task = () -> { ... }; // Geriye değer dönemez, Exception fırlatamaz!
   Callable<String> task = () -> "Sonuç"; // Değer döner, Checked Exception fırlatabilir!

3. MODERN JAVA 25 BUILDER PATTERN (JDK 21+ & 25):
   Thread.ofPlatform().name("worker-", 1).start(task);
   Thread.ofVirtual().name("loom-", 1).start(task);
   ✔ Tek bir arayüzle hem ağır OS thread'i hem de hafif Virtual Thread açabilme!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir kamyon almak istediğinizde şoförü de kamyonun motoruna kaynaklanmış olarak alırdınız (extends Thread). Artık kamyon başka (Thread), şoförün yapacağı iş emri başkadır (Runnable/Callable)! Runnable, 'Git depoyu süpür' diyen ama sana cevap vermeyen bir emirdir. Callable ise 'Kasada kaç para var say ve bana söyle' diyen, sana bir sonuç (Future) getiren emirdir. Java 25'te ise artık 'Bana 1 numaralı platform işçisini ver' veya 'Bana uçan sanal işçi ver' diyebildiğimiz harika bir sipariş menümüz (Thread.ofPlatform() / Thread.ofVirtual()) vardır!

🇬🇧 ENGLISH (ELI5)

In Java 1.0, to buy a delivery truck, you had to weld the human driver directly to the chassis (`extends Thread`). Modern engineering separates the vehicle (Thread) from the cargo assignment (Runnable/Callable). `Runnable` is a 'Fire-and-forget' chore returning `void`. `Callable\<V>` is an order returning an explicit computed value and capable of throwing checked exceptions. In modern Java 25, the fluent `Thread.ofPlatform()` and `Thread.ofVirtual()` builders allow seamlessly stamping out heavyweight OS threads or lightweight user-mode fibers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Java 25 Thread Builder Mimarisi: 1) `Thread` sınıfından türetmenin zararları: Object Oriented prensiplerine aykırıdır; 'İş' (Task) ile o işi 'Çalıştıran Motor' (Worker) birbirine sıkı sıkıya bağlanır. 2) `Runnable` vs `Callable`: Runnable `run()` metodu `void` döner ve checked exception fırlatamaz; Callable `call()` ise generik `V` döner ve `throws Exception` imzasına sahiptir. 3) Modern `Thread.Builder`: Java 21 ile gelen ve Java 25'te standartlaşan `Thread.ofPlatform()` ve `Thread.ofVirtual()`, thread konfigürasyonunu (isim öneki, daemon durumu, priority, UncaughtExceptionHandler) fluent API ile güvenle kurar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architectural Analysis of Thread Factories: 1) The Fallacy of Subclassing: Inheritance couples the task logic with OS thread scheduling state, violating the Single Responsibility Principle (SRP). 2) `Runnable` vs `Callable\<V>` Contract: `Runnable.run()` is a void-returning, non-throwing functional interface. `Callable\<V>.call()` enables return value propagation and checked exception handling, foundational to Executor frameworks. 3) Java 25 Fluent Builders: `Thread.ofPlatform()` and `Thread.ofVirtual()` provide type-safe builders for thread naming, daemon flags, uncaught exception handlers, and thread factory generation (`ThreadFactory`).

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Modern Thread Builders ve Callable Entegrasyonu
import java.util.concurrent.Callable;
import java.util.concurrent.FutureTask;

void main() throws Exception {
    // 1. Modern Java 25 Platform Thread Builder
    Thread platformThread = Thread.ofPlatform()
        .name("payment-worker-", 1)
        .daemon(false)
        .start(() -> IO.println("Platform thread çalıştı: " + Thread.currentThread()));
    platformThread.join();

    // 2. Callable ile değer döndüren iş parçacığı
    Callable<Integer> calculateTaxTask = () -> {
        IO.println("Vergi hesaplanıyor...");
        return 1850;
    };

    var futureTask = new FutureTask<>(calculateTaxTask);
    Thread.ofPlatform().name("tax-calc").start(futureTask);

    // get() çağrısı hesaplama bitene kadar bekler (Blocking)
    int result = futureTask.get();
    IO.println("Hesaplanan Vergi: " + result + " TL");
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: 'Thread.run() çağırmak ile Thread.start() çağırmak arasındaki fark nedir?' Yanıt: 'start() metodu JVM üzerinden işletim sistemine yeni bir thread tahsis ettirir (OS thread) ve o yeni thread üzerinde run() metodunu tetikler. Doğrudan run() çağrılırsa yeni thread açılmaz, kod o anki mevcut thread içinde senkron çalışır!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic Phone Screen Trap: 'What happens if you invoke `thread.run()` instead of `thread.start()`?' Answer: '`start()` prompts the JVM to allocate an underlying OS kernel thread and invoke `run()` asynchronously. Calling `run()` directly merely executes the method synchronously on the current caller's thread, bypassing multithreading entirely.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Thread sınıfını extend etmek yerine Runnable/Callable kompozisyonu kullanılmalıdır.
- Callable geriye değer dönebilir ve Exception fırlatabilir; Runnable dönemez.
- Java 25'te Thread.ofPlatform() ve Thread.ofVirtual() standart builder'dır.

🎯 Key Takeaways for the Interview

- Favor composition (Runnable/Callable) over inheriting from Thread.
- Callable\<V> yields return values and throws checked exceptions; Runnable returns void.
- Java 25 fluent builders Thread.ofPlatform() / ofVirtual() unify thread instantiations.